RUTrending: A Multi-Agent Framework for Automated Residence Life Trend Analysis

Jelani Beaugris Sameer Jiandani Srimathi Vadivel
Rutgers University–New Brunswick

github.com/JelaniB0/RUTrending ru-trend.vercel.app

***Abstract*—This paper presents RUTrending, a modernized residence life reporting framework for Rutgers University. The system incorporates a backend multi-agent pipeline for automated trend analysis and synthetic data generation via few-shot prompting, enabling campus-wide and building-wide trend analysis that is more accessible to higher-level Residence Life staff than existing manual approaches. Everything in the GitHub codebase was generated by Claude Code**

1. Introduction

Residence Life Staff at Rutgers University generates several hundred to thousands of incident reports each semester. Despite this large influx of valuable data, identifying campus-wide or building-wide trends remains a largely manual, time-consuming task for higher-level staff, such as Assistant Residence Life Coordinators and Residence Life Coordinators. This is mainly due to a lack of access to technology that could make that process easier for them; instead, they have to rely either on higher-ups compiling the information first and communicating the trends to them, or on eyeballing trends based on what they've observed. Both approaches have clear limitations. Waiting for a compiled trend report can delay response to issues that require timely intervention. Meanwhile, manually eyeballing trends is unreliable, particularly when a single Residence Life Coordinator may oversee hundreds of reports per semester.

The overall goal of the framework is to provide Residence Life Staff with an easy-to-use interface for gaining targeted insights and quickly identifying trends. With faster access to trend data, staff can allocate resources more effectively and respond with well-informed intervention strategies. This work investigates whether a specialized multi-agent LLM architecture can automate trend analysis over structured institutional report data more effectively than manual or single-agent approaches. The primary contributions of this work are:

- A multi-agent LLM pipeline that performs parallel, specialized trend analysis over structured incident report data
- A few-shot prompting methodology for generating realistic synthetic residence life reports to address data scarcity

- A full-stack web interface enabling non-technical Residence Life staff to query and visualize trends without manual data aggregation
- An empirical evaluation comparing multi-agent versus single-agent approaches for institutional report analysis

2. Related Work

Certain technologies used in this framework, such as AI agents, have proven to excel at tasks like trend analysis. AI agents have demonstrated strong performance in research and summarization tasks, with recent industry surveys identifying these as the most common production use cases for agentic systems [1]. In general, research has shown that multi-agent orchestration significantly outperforms single-agent approaches for analysis tasks, with studies demonstrating that specialized agent architectures produce substantially more accurate and actionable outputs than single-agent systems [2].

Recent surveys on LLM-based multi-agent systems have characterized the core design challenges in this space, including task allocation, agent communication, and shared state management [3, 4]. These works highlight that multi-agent systems are particularly effective when tasks can be decomposed into specialized subtasks — the central design principle behind our pipeline's parallel agent execution and conditional routing architecture.

The evaluation of LLM-generated outputs presents its own methodological challenges. The LLM-as-a-Judge paradigm has emerged as a scalable alternative to human evaluation, with studies demonstrating strong alignment between LLM judges and human assessments across a range of dimensions including accuracy, relevance, and completeness [5, 6]. Our evaluation methodology adopts this approach, combining a blind LLM judge with programmatic accuracy scoring to mitigate the limitations of either method alone.

Few-shot prompting has also proven effective for synthetic data generation, particularly in low-resource settings where real labeled data is scarce or sensitive [7], directly motivating our approach to dataset construction.

3. Methodology

3.1. System Architecture

The framework consists of two primary components: a Next.js frontend and a backend multi-agent pipeline. The frontend provides Residence Life staff with an intuitive dashboard for submitting reports, filtering trends by campus and date range, and viewing AI-generated insights. It is built using Next.js, React, and Recharts for data visualization, and is organized around four main views: a report submission interface, an analytics dashboard, a map view, and an AI analysis panel.

The backend is responsible for data persistence and AI-driven analysis. Report data is stored in a PostgreSQL database and accessed through Prisma ORM. Both components communicate via a REST API, exposing a single /api/query endpoint that accepts campus, date range, and optional natural-language query parameters. The internal design of the backend pipeline is described in Section 3.2.

3.2. Framework Architecture

The backend pipeline is designed around a directed acyclic graph (DAG) implemented using LangGraph, a framework for building stateful multi-agent workflows. The graph consists of eight nodes, each representing a specialized agent or routing mechanism, connected by directed edges that specify execution order, data flow, and agent communication. Upon receiving a request, the pipeline initializes a shared state object containing the query parameters, which is passed through the graph and progressively enriched by each agent's output.

A key architectural decision made was the use of parallel agent execution for the four core analysis agents: Building, Campus, Alert, and the Query agent. Following the Intake Agent's data retrieval phase, the four agents execute simultaneously, with each analyzing a distinct dimension of the incident report data. This fan-out strategy reduces latency compared to sequential execution, as multiple agents can run concurrently and return results, rather than waiting for one agent to finish before executing the next. Two additional agents are conditionally triggered based on routing flags set during parallel execution, as detailed in Section 3.3.

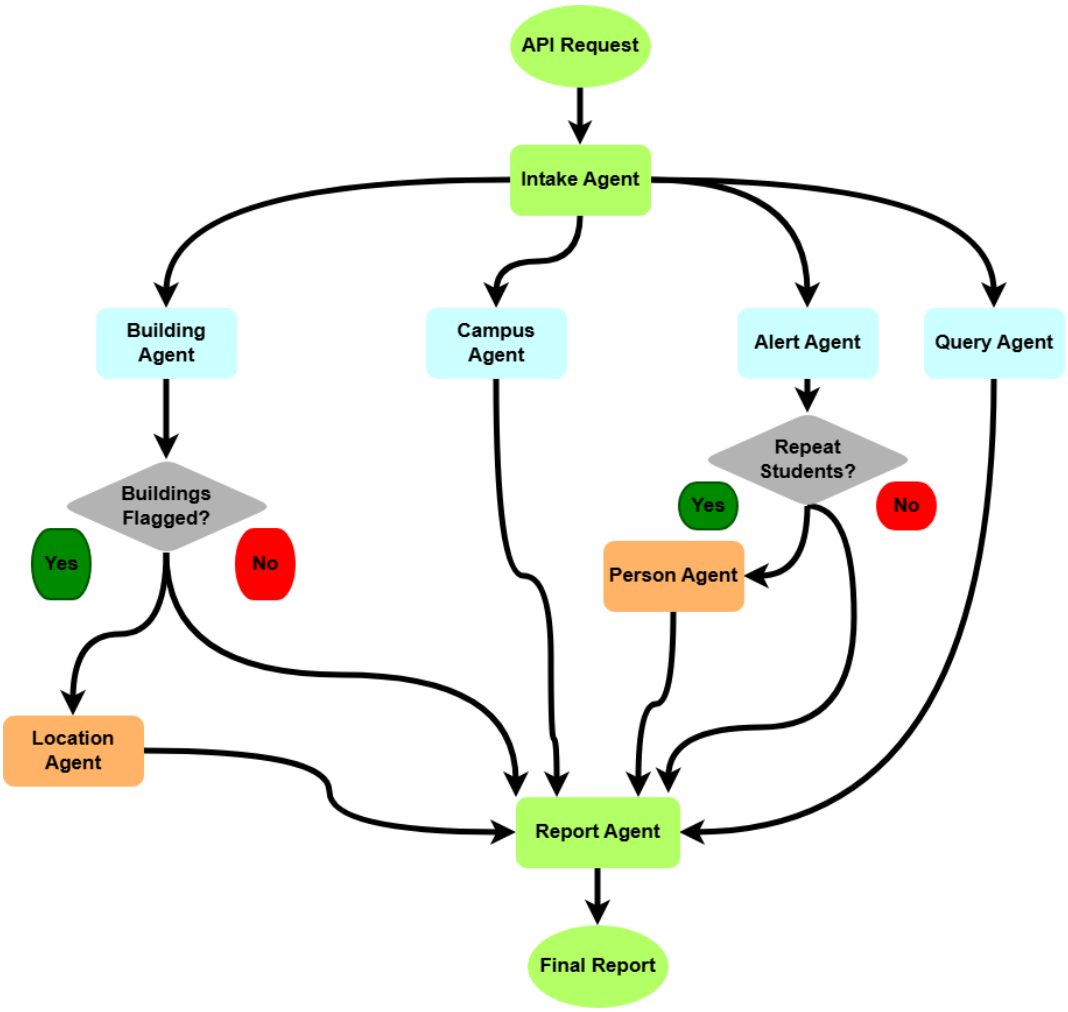

Figure 1: Directed acyclic graph framework architecture.

3.3. Agent Descriptions

The pipeline consists of eight specialized agents organized into four execution phases. Table 1 provides a high-level overview of each agent's role and the model used. All agents communicate through a shared state object (`TrendState`) that is initialized at request time and progressively enriched as the graph executes. The state stores raw data payloads, agent outputs, and routing flags that govern conditional execution. Two key flags — `shouldDrillLocation` and `shouldDrillPerson` — determine whether the Location and Person agents are invoked during a given request, based on patterns detected during parallel analysis.

Table 1: Agent descriptions and execution behavior.

Agent	Type	Role	Model
Intake	Core	Fetches raw reports, building stats, and campus stats from the database	GPT-4.1
Building	Analysis	Analyzes per-building incident patterns and flags buildings for drill-down	GPT-4.1
Campus	Analysis	Performs campus-wide trend analysis across all four Rutgers campuses	GPT-4.1
Alert	Analysis	Scans for urgent patterns including repeat locations and high RUPD/EMS involvement	GPT-4.1
Query	Analysis	Translates natural language questions into SQL and returns targeted answers	GPT-4.1
Location	Conditional	Drills into flagged buildings at room and floor level to identify hotspots	GPT-4o-mini
Person	Conditional	Analyzes students appearing in multiple reports for escalation and support needs	GPT-4.1
Report	Core	Compiles all agent outputs into a structured executive summary	GPT-4.1

Phase 1 — Data Acquisition. The *Intake Agent* serves as the entry point of the pipeline. It fetches all relevant incident reports from the database based on the campus and date filters provided in the request, and executes aggregated SQL queries to populate `buildingStats` and `campusStats` in the shared state. These payloads serve as the ground-truth data source for all downstream agents.

Phase 2 — Parallel Macro Analysis. Four agents execute simultaneously following data acquisition. The *Building Agent* analyzes the top 10 buildings by incident volume, identifying high-rate buildings and concerning patterns such as elevated Title IX or mental health reports. It outputs a `FLAGGED_BUILDINGS` array of buildings exceeding four reports or exhibiting alarming incident concentrations, which sets the `shouldDrillLocation` routing flag. The *Campus Agent* performs broad campus-wide trend analysis, focusing on macro-level differences between Rutgers campuses and high-level resource prioritization. The *Alert Agent* scans for high-severity markers including RUPD and EMS involvement, repeat incident locations, and students appearing in multiple reports. It generates a structured array of actionable `ALERT:` strings and sets the `shouldDrillPerson` flag when repeat students are detected. The *Query Agent* handles

arbitrary natural language questions from staff by translating them into PostgreSQL queries via a text-to-SQL prompt, executing the query against the database, and formatting the result into a direct, data-grounded answer.

Phase 3 — Conditional Micro Analysis. Two agents are conditionally triggered based on routing flags set during Phase 2. The *Location Agent* fires when `shouldDrillLocation` is true. It queries the database for specific rooms, floors, and common areas within flagged buildings, groups incidents by location, and identifies multi-incident hotspots warranting localized intervention. The *Person Agent* fires when `shouldDrillPerson` is true. It retrieves all students appearing in two or more reports and clinically evaluates escalation risks, distinguishing between students appearing as accused parties versus victims or students of concern. Student privacy is preserved by referencing individuals by initials only in all generated output.Once all parallel and conditional branches have resolved, the *Report Agent* then merges outputs from all upstream agents alongside strict ground-truth numerical data from `campusStats` and `buildingStats`. To prevent hallucination, the Report Agent is explicitly instructed to cite only figures present in the ground-truth data, ignoring any numbers carried over from intermediate agent analyses. The final output is a structured 4–6 paragraph executive summary suitable for Residence Life directors.

3.4. Data Collection & Synthetic Generation

Real incident report data from Rutgers Residence Life was not available for use in this project due to accessibility and privacy constraints. To address this, a synthetic dataset of 1,150 incident reports was generated using few-shot prompting with Claude Sonnet 4.6.

Several anonymized example reports were manually authored to reflect the structure and language typical of real Residence Life incident documentation, while also adding additional context to the prompt, including report types, common patterns in incident report writing, and the overall style of incident reports. This included fields such as incident nature, policy type, specific location, RUPD and EMS involvement, student role, and reslife staff role. These examples were provided to the model as few-shot prompts alongside the instructions to generate realistic variations across all four Rutgers campuses, the 45 residential buildings, and the full range of incident natures and policy types defined in the system schema. The model was instructed to realistically vary incident frequency distributions, reflecting that policy violations and general concerns are more common than Title IX or crisis-level incidents, and to randomly distribute reports across buildings.

The resulting dataset of 1,150 reports was inserted directly into the PostgreSQL database and used as the ground truth for all pipeline evaluation and frontend demonstrations.

Example Generation Prompt

You are generating synthetic Rutgers University Residence Life incident reports for research purposes. Below is an example report. Generate 200 realistic reports in JSON format...
On Sunday, November 2, 2025 at 2:45pm, Resident Assistant John Doe was conducting a health and safety inspection in Thomas Suites and observed a hoverboard with a lithium-ion battery plugged in in 213th Floor Hallway. Resident Leila Rodriguez was present. John informed Leila that transportation devices with rechargeable lithium-ion batteries are prohibited under the Fire Safety Policy due to the risk of fire. John instructed Leila to immediately unplug and remove the device from the building. Leila cooperated and removed the device. John documented the violation and notified ARLC David Ortega at 3:00pm. Documentation was completed at 3:30pm. End of report.

4. Experiments & Results

4.1. Experiment 1: Multi-Agent vs. Single-Agent Baseline

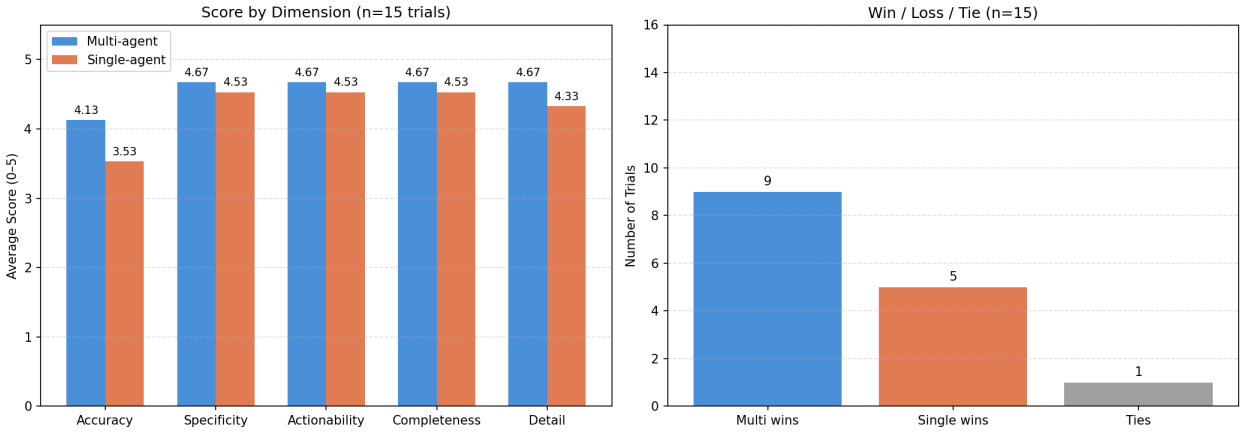

Figure 2: Average scores across 5 evaluation dimensions over 15 trials. Accuracy was scored programmatically; remaining dimensions were scored by a blind LLM judge.

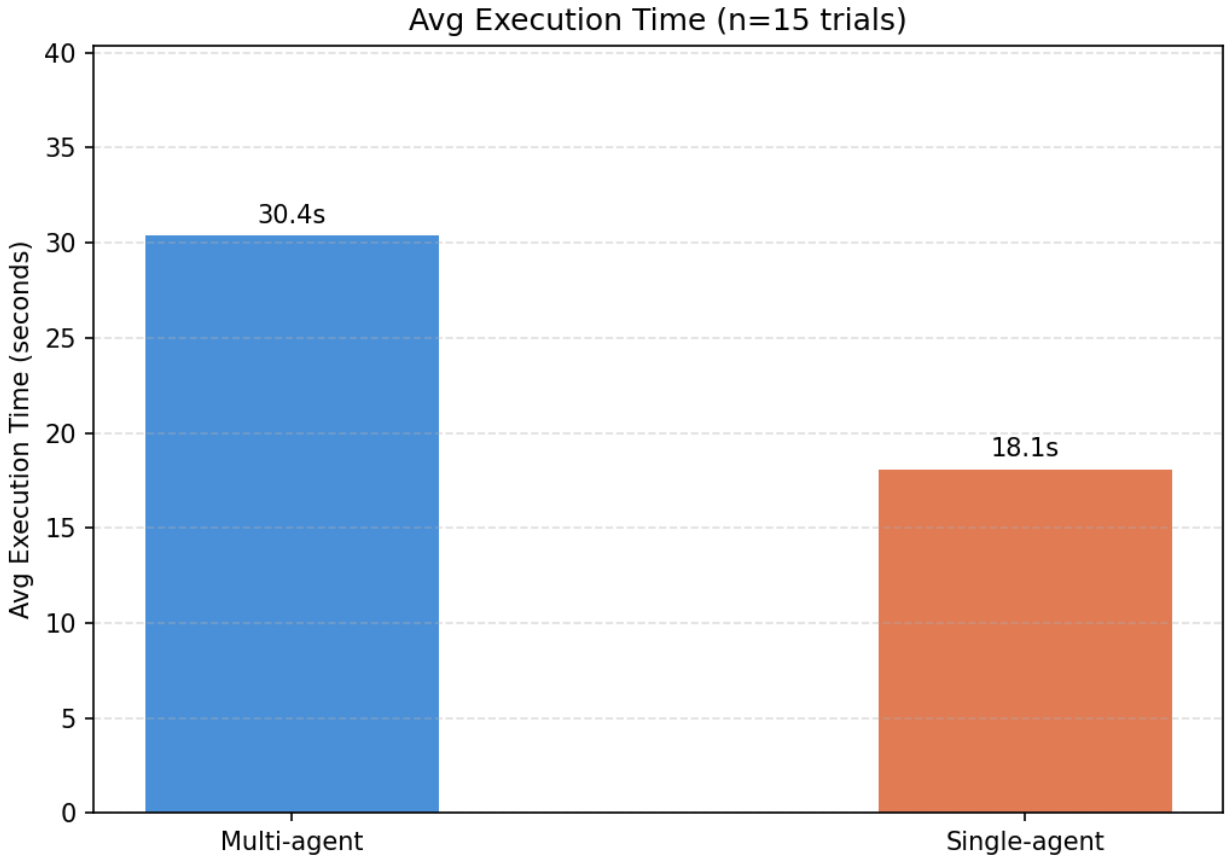

Figure 3: Average execution time (n=15).

Figure 2 summarizes the results across 15 trials. The multi-agent pipeline outperformed the single-agent baseline in 9 of 15 trials, with 5 single-agent wins and 1 tie. The most significant advantage was in accuracy (+0.60), which was scored programmatically by cross-referencing cited statistics against ground-truth database values, making it the most reliable metric in the evaluation. The detail dimension showed the second largest gap (+0.33), consistent with the multi-agent system's location drill-down capability producing floor and room-level specifics unavailable to the single-agent baseline.

Qualitative dimensions, specificity, actionability, and completeness, were nearly equivalent between the two approaches (0.13 advantage each), suggesting that the single-agent system is capable of producing well-structured, articulate prose when given sufficient data. The multi-agent advantage lies not in writing quality but in factual grounding and analytical depth.

According to figure 3, the multi-agent pipeline averaged 30.4 seconds per request, compared to 18.1 seconds for the single-agent baseline, resulting in a 1.68x increase in latency on average. For the intended use case of periodic trend review by Residence Life staff, this trade-off is acceptable, especially since trend analysis occurs on occasion and is not a continuously recurring process.

4.2. Experiment 2: Ablation Study

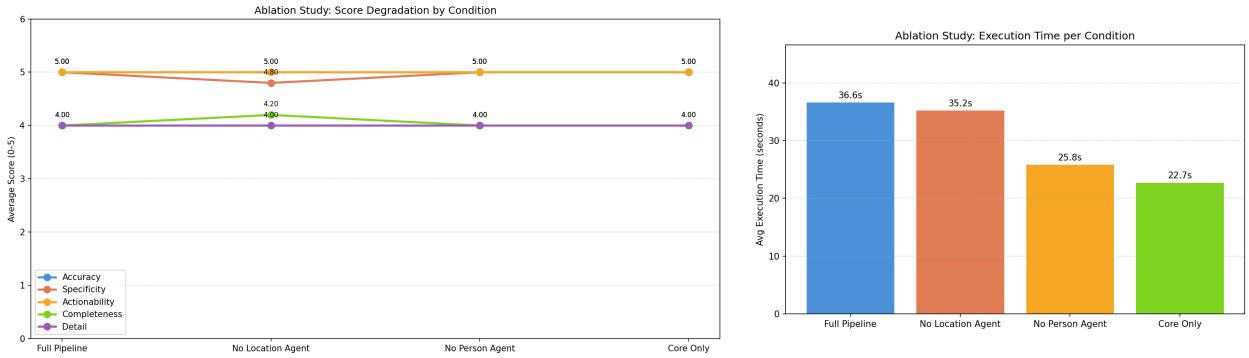

Figure 4: Left: score degradation by condition across 5 evaluation dimensions (5 trials per condition). Right: average execution time per condition.

Figure 4 presents the results of an ablation study examining the contribution of the two conditional agents, Location and Person, across four pipeline configurations. Quality scores remained largely resilient across conditions, with the most notable degradation occurring in specificity when the Location Agent was removed (5.00 $\rightarrow$ 4.80), consistent with the loss of floor and room-level hotspot data that agent provides.

The more pronounced effect of ablation was observed in execution time. Removing the Person Agent reduced average latency by 10.8 seconds (36.6s $\rightarrow$ 25.8s), while removing the Location Agent produced only a marginal reduction of 1.4 seconds. This asymmetry suggests that the Person Agent is computationally expensive relative to its measurable quality contribution, whereas the Location Agent provides targeted specificity gains at minimal additional cost. The Core Only condition achieved the lowest latency (22.7s) while maintaining scores within 0.20 points of the full pipeline across all dimensions, indicating that the core agents are responsible for the majority of output quality.

These results validate the conditional routing design: by dynamically triggering the Location and Person agents only when warranted by routing flags, the system achieves full-pipeline quality in the common case while avoiding unnecessary latency when drill-down analysis is not needed.

5. Limitations

Several limitations of this work should be acknowledged. First, due to the non-deterministic nature of LLM inference, individual trial scores exhibit variance across runs. While results are reported as averages across multiple independent runs to mitigate this effect, exact figures may not be fully reproducible. The directional findings, multi-agent outperforming single-agent, and conditional agents contributing proportional value, are consistent across runs, but the precise score margins should be interpreted with this variance in mind.

Second, resource and time constraints limited the scale of evaluation. Experiment 1 was conducted over 15 trials (3 runs of 5) and Experiment 2 over 5 trials per condition, which, while sufficient to establish directional trends, may not be fully representative of the pipeline's behavior across a broader range of query types, campus configurations, or edge case inputs. A more comprehensive evaluation would include a larger trial set and a wider diversity of natural language queries.

Third, the multi-agent architecture introduces practical constraints at the infrastructure level. Each pipeline invocation consumes a significant number of LLM tokens across eight agents, with the Report Agent alone receiving the aggregated outputs of all upstream agents as context. This creates token reservation pressure that could become a bottleneck at scale or when processing large date ranges with high report volumes, or when processing reports with high narrative density. Additionally, while parallel execution mitigates latency, the full pipeline still averages 36.6 seconds per request, a tradeoff that is acceptable for periodic trend review but would require architectural optimization for any real-time or high-frequency use case.

Finally, the evaluation was conducted entirely on a synthetic dataset. While the generation methodology was designed to reflect realistic distributions, real incident report data may contain noisier language, inconsistent formatting, and edge cases not captured in the synthetic generation process, which could affect pipeline performance in deployment.

6. Conclusion

This paper presented RUTrending, which includes a backend multi-agent LLM pipeline for automated trend analysis over structured institutional incident report data. The system addresses a real operational gap in Rutgers Residence Life, where identifying campus-wide and building-wide trends has historically required manual effort that is both time-consuming and unreliable at scale.

Across both experiments, the multi-agent pipeline demonstrated superior accuracy (+0.60) and detail (+0.33) over the single-agent baseline, while the ablation study confirmed that conditional routing is well-calibrated, delivering full-pipeline quality while avoiding unnecessary latency when drill-down agents are not warranted.

Future work could explore fine-tuning smaller models on real anonymized report data, extending the pipeline to support longitudinal trend comparison across semesters, and deploying the system with actual Residence Life staff to evaluate real-world usability and impact. Additionally, further research into optimizing token and agent call efficiency without degrading output quality would improve the framework's cost-effectiveness and scalability in production environments.

References

[1] LangChain, "State of AI agents," https://www.langchain.com/stateofaiagents, 2024, accessed: 2026-04-22.

[2] P. Drammeh, "Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response," 2025.

[3] T. Guo *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 8048–8057.

[4] S. Han *et al.*, "LLM multi-agent systems: Challenges and open problems," 2024.

[5] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] H. Li *et al.*, "LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods," 2024.

[7] M. Nadas, L. Diosan, and A. Tomescu, "Synthetic data generation using large language models: Advances in text and code," 2025.